BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 015 · Noughts & Crosses · Part 3 — Make moves

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Which method from Part 2 displays the board?

Q2. In a zero-indexed C# 2D array, which row and column does board[1,2] access?

Q3. What do nested loops let us do with the board?

Q4. Move 5 is the centre square. What zero-based row and column should it map to?

Do Now Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — 2D array

Recall the term — Row

Recall the term — Column

Recall the term — Nested loop

Recall the term — Validation

Recall the term — Game loop

Recall the term — Method

New learning

Map 1–9 into row/column — CODE:
int row = (move - 1) / 3;
int col = (move - 1) % 3; — Integer division finds the row; MOD finds the column. The picture highlights move 8: row 2, column 1. Inside MakeMove(), check the 1–9 range before using these indexes.

Check the move before accessing the array — CODE:

```
if (move < 1 || move > 9)
return false;

// Now calculate row and col.
// Only access the array for a valid index.
if (board[row,col] == "X" ||
board[row,col] == "O")
{
return false;
```

} — Reject choices outside 1–9 before calculating/using their indexes. Reject occupied squares too. Never overwrite an X or O.

Make the move — CODE:

```
board[row,col] = player;
return true;
```

— A Boolean return value tells Main whether the move succeeded.

Switch player only after success — CODE:

```
if (currentPlayer == "X")
currentPlayer = "O";
else
currentPlayer = "X";
```

— An invalid move should leave the same player to try again.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Worked example

Turn move 8 into a board position

1. Subtract 1 — CODE:
move - 1 = 7 — Board indexes begin at 0.

2. Integer divide by 3 — CODE:
7 / 3 = 2
row = 2 — Integer division discards the remainder.

3. Use MOD 3 — CODE:
7 % 3 = 1
col = 1 — The remainder identifies the column.

4. Place the player — CODE:
board[2,1] = player; — Move 8 maps to row 2, column 1.

We Do · Practise with guidance

Map a move

Move 6 is chosen.

Complete the same calculation

```
row = (6 - 1) / 3 = ?
col = (6 - 1) % 3 = ?
```

```
row = (6 - 1) / 3 = 1
col = (6 - 1) % 3 = 2
```

Validate range

Reject choices outside 1–9.

Fill the same condition

```
if (move < ? || move > ?)
{
    return false;
}
```

```
if (move < 1 || move > 9)
{
    return false;
}
```

Reject occupied square

The target may already contain X or O.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

```
if (board[row,col] == ? ||
    board[row,col] == ?)
{
    return false;
}

if (board[row,col] == "X" ||
    board[row,col] == "O")
{
    return false;
}
```

Switch player

Return the other symbol.

Fill the same code

```
if (player == "X")
    return ?;
else
    return ?;
```


BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

```
public static string SwitchPlayer(string player)
{
if (player == "X")
return "O";
else
return "X";
}
```

Check: The method alternates correctly.

Q4. Make alternating turns work

CODE:

```
string currentPlayer = "X";
int movesMade = 0;

while (movesMade < 9)
{
DrawBoard();
Console.Write("Player " + currentPlayer + ", choose 1-9: ");
int move = int.Parse(Console.ReadLine());

if (MakeMove(move, currentPlayer))
{
movesMade++;
currentPlayer = SwitchPlayer(currentPlayer);
}
else
{
Console.WriteLine("Invalid move - try again.");
}
}

DrawBoard();
```

Check: Checkpoint reached: X and O can take valid alternating turns. Win/draw detection comes next lesson.

Exit Ticket · Model answer

False. An occupied square is invalid. Ask the same player to choose another square.

Try the question again

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Map a move

Write code that converts an integer move from 1–9 into zero-based row and column values.

Question 2

Write MakeMove()

Create MakeMove(int move, string player) . Reject out-of-range and occupied squares; otherwise place the symbol and return true.

Question 3

Write SwitchPlayer()

Return O when given X, otherwise return X.

Question 4

Make alternating turns work

Use your Part 2 DrawBoard() , prompt for a move, call MakeMove() , and switch player only after a valid move.

Exit Ticket · Answer before checking

If a player chooses an occupied square, the program should switch to the other player anyway.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. DrawBoard().

Q2. Row 1, column 2.

Q3. Traverse every row and column.

Q4. [1,1].

You Do pages · Independent questions

Q1. Map a move
CODE:
int row = (move - 1) / 3;
int col = (move - 1) % 3;
Check: 1→[0,0], 5→[1,1], 9→[2,2].

Q2. Write MakeMove()
CODE:
public static bool MakeMove(int move, string player)
{
if (move < 1 || move > 9)
return false;

int row = (move - 1) / 3;
int col = (move - 1) % 3;

if (board[row,col] == "X" || board[row,col] == "O")
return false;

board[row,col] = player;
return true;
}
Check: Invalid moves leave the board unchanged.

Q3. Write SwitchPlayer()
CODE: